

Programmation & Algorithmique 2

TP - Séance 6

9 avril 2025

Matière visée : Héritage et exceptions, Package et Streams (Lecture/écriture dans fichiers 'texte', et lecture/écriture de types primitifs dans fichiers binaires).

Dans les exercices suivants, vous devrez tester vos solutions avec des fichiers de test. Un "gros" fichier de test textuel est disponible sur Moodle : `scottish_parliament_official_reports.txt`, pesant 2.4MB. Vous pouvez, bien-sûr, tester avec vos propres fichiers. Portez tout de même une attention particulière aux exceptions qui pourraient survenir, qu'il s'agisse de problèmes de fichiers introuvés, de mauvaise lecture, ... **Ces problèmes doivent être gérés via des exceptions.**

1 Héritage

1.1 Gestion d'une bibliothèque

On veut modéliser la gestion d'une bibliothèque : on définira un certain nombre de classes : `Main`, `Ouvrage`, `BiblioTab`, `Bibliotheque`, `Periodique`, `CD`, `Livre`. Les livres auront comme caractéristiques : auteur, titre, éditeur ; les périodiques : nom, numéro, périodicité ; les CDs : titre, auteur. De plus tous les ouvrages auront une date d'emprunt (potentiellement nulle), une cote (le numéro par ordre de création). On implémentera également sur chaque objet une méthode `toString()` renvoyant toutes les informations sur l'ouvrage sous forme d'une chaîne de caractères.

La classe `BiblioTab` permettra de stocker dans une structure les Ouvrages (ajout et suppression, la suppression prenant en argument la cote de l'ouvrage). Elle aura également une méthode `toString()` affichant le nombre d'ouvrages, puis chaque ouvrage successivement. La classe `Bibliotheque` sera simplement une version abstraite déclarant les mêmes méthodes que `BiblioTab` mais sans les implémenter. `BiblioTab` héritera de `Bibliotheque`.

La classe `Main` ne contiendra que la méthode `main` et testera la bibliothèque en y insérant et supprimant quelques ouvrages, puis en affichant le contenu de la bibliothèque.

1. Représentez les différentes classes dans un graphe d'héritage. On mettra en évidence pour chaque classe les méthodes et les champs qu'elle définit, redéfinit ou hérite. On souhaite que tous les champs soient déclarés privés et que l'on ne puisse y accéder de l'extérieur que par des méthodes ; et
2. Implémentez les classes ci-dessus. Pour la classe `BiblioTab` on utilisera un tableau de longueur suffisante (on vérifiera quand même à chaque opération d'insertion que l'on ne dépasse pas les bornes du tableau). Quel sont les inconvénients de cette méthode ?

Dans ce qui suit, on veut implémenter une deuxième version de la bibliothèque, que l'on appellera `BiblioList` et qui héritera également de `Bibliotheque`. Cette nouvelle implémentation utilisera la classe `ArrayList`.

1. Modifiez le minimum de choses dans la classe `Main` pour permettre l'utilisation de `BiblioList` ; et

2. Implémentez, à l'aide des `ArrayLists`, la classe `BiblioList`.

2 Lecture/écriture dans fichiers texte

Veuillez créer un package `be.ac.umons.info.textfile`.

2.1 Exercice 1

1. Créez une classe `ToUpper` que vous placerez dans ce package.
On doit exécuter cette classe en donnant en paramètres (à la ligne de commande) deux chaînes de caractères. Celles-ci symboliseront le nom du fichier d'entrée (dans lequel on lira), et le nom du fichier de sortie (dans lequel on écrira).
Le rôle de cette classe sera de lire le contenu du fichier d'entrée ligne par ligne, et de recopier ces lignes *en majuscule* dans le fichier de sortie.
2. Testez votre classe avec le fichier d'entrée de test.

2.2 Exercice 2

1. Créez une classe `FrequencyAnalyser` que vous placerez dans le package `be.ac.umons.info.textfile`. On doit pouvoir exécuter cette classe en donnant en paramètre (à la ligne de commande) une chaîne de caractères qui représentera le nom d'un fichier d'entrée. Le rôle de cette classe va être de calculer le pourcentage d'occurrences de chaque caractère dans le fichier d'entrée. Il ne faut considérer que les "lettres", c'est-à-dire les 26 lettres de l'alphabet en majuscules et minuscules.
À la fin de l'analyse, affichez à l'écran la fréquence d'occurrence de chaque caractère, triés par fréquences décroissantes.
2. Testez votre classe avec le fichier d'entrée de test ainsi qu'avec ce même fichier converti en majuscule.

Remarque : un caractère peut être considéré comme un nombre entier : son numéro dans sa représentation en ASCII par exemple.

3 Lecture/écriture dans fichiers binaires

Créez un package `be.ac.umons.info.binaryfile`.

3.1 Exercice 1

Créez un programme `Squared` que vous placerez dans le package `be.ac.umons.info.binaryfile`. Ce programme va stocker dans un fichier binaire les carrés ($i \times i$) des 1000 premiers naturels ($0 \leq i < 1000$). Il vous est demandé de n'utiliser que des données de type `int` pour réaliser cet exercice.

3.2 Exercice 2

Créez un programme `SquaredTest` qui prend en argument un fichier et un nombre naturel n . Le programme doit aller lire dans le fichier binaire le carré de ce nombre et l'afficher à l'écran.